

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №2
«Программирование контроллера прерываний»

Выполнил:
Студент группы 150503
Шарай П. Ю.

Проверил:
к. т. н., доцент
Одинец Д.Н.

Минск, 2023

1. Постановка задачи

Написать резидентную программу, выполняющую перенос всех векторов аппаратных прерываний ведущего и ведомого контроллера на пользовательские прерывания. При этом необходимо написать обработчики аппаратных прерываний, которые будут установлены на используемые пользовательские прерывания и будут выполнять следующие функции:

1. Выводить на экран в двоичной форме следующие регистры контроллеров прерывания (как ведущего, так и ведомого):

- регистр запросов на прерывания;
- регистр обслуживаемых прерываний;
- регистр масок.

При этом значения регистров должны выводиться всегда в одно и то же место экрана.

2. Осуществлять переход на стандартные обработчики аппаратных прерываний, для обеспечения нормальной работы компьютера.

Базовый вектор - 80H / 08H.

2. Алгоритм

Для переноса векторов прерываний ведущего и ведомого векторов контроллера на пользовательские прерывания используются функции `getvect` и `setvect`.

Инициализация контроллеров осуществляется посредством вызова последовательности команд ICW1, ICW2, ICW3 и ICW4.

Функция `_dos_keep` обеспечивает выход в DOS и оставляет программу резидентной.

3. Листинг программы

```
#include <dos.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

#define COLOR_COUNT 4

struct VIDEO
{
    unsigned char symbol;
    unsigned char attribute;
};

unsigned char colors[COLOR_COUNT] =
{0x71, 0x62, 0x54, 0x26};
char color = 0x87;
```

```

void changeColor()
{
    color = colors[rand() % COLOR_COUNT];
    return;
}

void print() // Функция для показа
на экране состояний регистров (запросов, маски, обслуживаемых
запросов)
{
    char temp;
    int i, val;
    VIDEO far* screen = (VIDEO far *)MK_FP(0xB800, 0);

    val = inp(0x21); // получаем регистр
маски ведущего
    for (i = 0; i < 8; i++)
    {
        temp = val % 2;
        val = val >> 1;
        screen->symbol = temp + '0';
        screen->attribute = color;
        screen++;
    }
    screen++;

    val = inp(0xA1); // получаем
регистр маски ведомого
    for (i = 0; i < 8; i++)
    {
        temp = val % 2;

        val = val >> 1;

        screen->symbol = temp + '0';
        screen->attribute = color;
        screen++;
    }
    screen += 63;
    outp(0x20, 0x0A);

    val = inp(0x20); // получаем
регистр запросов ведущего
    for (i = 0; i < 8; i++)
    {
        temp = val % 2;

        val = val >> 1;

        screen->symbol = temp + '0';
        screen->attribute = color;
        screen++;
    }
    screen++;

    outp(0xA0, 0x0A);
}

```

```

        val = inp(0xA0);                                // получаем
регистр запросов ведомого
        for (i = 0; i < 8; i++)

        {
            temp = val % 2;

            val = val >> 1;

            screen->symbol = temp + '0';
            screen->attribute = color;
            screen++;

        }
        screen+=63;

        outp(0x20,0x0B);
        val = inp(0x20);                                // получаем
регистр обслуживаемых запросов ведущего
        for (i = 0; i < 8; i++)
        {
            temp = val % 2;

            val = val >> 1;

            screen->symbol = temp + '0';
            screen->attribute = color;
            screen++;

        }
        screen++;

        outp(0xA0,0x0B);
        val = inp(0xA0);                                // получаем
регистр обслуживаемых запросов ведущего
        for (i = 0; i < 8; i++)
        {
            temp = val % 2;

            val = val >> 1;

            screen->symbol = temp + '0';
            screen->attribute = color;
            screen++;

        }
    }

// IRQ 0-7

void interrupt(*oldint8) (...);                        // IRQ 0 - прерывание
таймера (18,2 times per second)
void interrupt(*oldint9) (...);                        // IRQ 1 - прерывание
клавиатуры
void interrupt(*oldint10) (...);                       // IRQ 2 - прерывание for
cascade interruptions in AT machines
void interrupt(*oldint11) (...);                      // IRQ 3 - прерывание COM 2
void interrupt(*oldint12) (...);                      // IRQ 4 - прерывание COM 1
void interrupt(*oldint13) (...);                      // IRQ 5 - прерывание
контроллера жесткого диска

```

```

void interrupt(*oldint14) (...);          // IRQ 6 - прерывание флорпи
диска (по закончению работы с ним)
void interrupt(*oldint15) (...);          // IRQ 7 - прерывание
принтера

// IRQ 8-15

void interrupt(*oldint70) (...);           // IRQ 8 - прерывание часов
реального времени
void interrupt(*oldint71) (...);           // IRQ 9 - резервирование
прерывание
void interrupt(*oldint72) (...);           // IRQ 10 - резервирование
прерывание
void interrupt(*oldint73) (...);           // IRQ 11 - резервирование
прерывание
void interrupt(*oldint74) (...);           // IRQ 12 - резервирование
прерывание
void interrupt(*oldint75) (...);           // IRQ 13 - прерывание
математического сопроцессора
void interrupt(*oldint76) (...);           // IRQ 14 - прерывание
жесткого диска
void interrupt(*oldint77) (...);           // IRQ 15 - резервирование
прерывание

void interrupt newint08(...) { print(); oldint8(); }
void interrupt newint09(...) { changeColor(); print(); oldint9(); }
void interrupt newint0A(...) { changeColor(); print(); oldint10(); }
void interrupt newint0B(...) { changeColor(); print(); oldint11(); }
void interrupt newint0C(...) { changeColor(); print(); oldint12(); }
void interrupt newint0D(...) { changeColor(); print(); oldint13(); }
void interrupt newint0E(...) { changeColor(); print(); oldint14(); }
void interrupt newint0F(...) { changeColor(); print(); oldint15(); }

void interrupt newintD0(...) { changeColor(); print(); oldint70(); }
void interrupt newintD1(...) { changeColor(); print(); oldint71(); }
void interrupt newintD2(...) { changeColor(); print(); oldint72(); }
void interrupt newintD3(...) { changeColor(); print(); oldint73(); }
void interrupt newintD4(...) { changeColor(); print(); oldint74(); }
void interrupt newintD5(...) { changeColor(); print(); oldint75(); }
void interrupt newintD6(...) { changeColor(); print(); oldint76(); }
void interrupt newintD7(...) { changeColor(); print(); oldint77(); }

void initialize()
{
    oldint8 = getvect(0x08);
    oldint9 = getvect(0x09);
    oldint10 = getvect(0x0A);
    oldint11 = getvect(0x0B);
    oldint12 = getvect(0x0C);
    oldint13 = getvect(0x0D);
    oldint14 = getvect(0x0E);
    oldint15 = getvect(0x0F);

    oldint70 = getvect(0x70);
    oldint71 = getvect(0x71);
    oldint72 = getvect(0x72);
    oldint73 = getvect(0x73);

```

```

oldint74 = getvect(0x74);
oldint75 = getvect(0x75);
oldint76 = getvect(0x76);
oldint77 = getvect(0x77);

//IRQ 0-7
setvect(0xD0, newint08);
setvect(0xD1, newint09);
setvect(0xD2, newint0A);
setvect(0xD3, newint0B);
setvect(0xD4, newint0C);
setvect(0xD5, newint0D);
setvect(0xD6, newint0E);
setvect(0xD7, newint0F);

//IRQ8-15
setvect(0x08, newintD0);
setvect(0x09, newintD1);
setvect(0x0A, newintD2);
setvect(0x0B, newintD3);
setvect(0x0C, newintD4);
setvect(0x0D, newintD5);
setvect(0x0E, newintD6);
setvect(0x0F, newintD7);

_disable(); // Запретить прерывания
20h/A0 - ICW1 (для записи);
21h/A1 - ICW2, ICW3, ICW4 (для записи)
//OCW - команда контроллера для
управления, ICW - команда контроллера для инициализации
outp(0x20, 0x11); //ICW1 - инициализация ведущего
контроллера (000 1 0 1) 0
//
должен быть режим срабатывания размер вектора
использование ведомого ICW4
//
установлен в 1 триггера (по фронту) прерывания (8 бит)
контроллера (0 - используется) будет использоваться
outp(0x21, 0x70); //ICW2 - инициализация ведущего
контроллера
(01110 000)
// установка адреса вектора прерывания
для первого контроллера нужно значение 08h
используются не используются
// для IRQ0 или IRQ8
для второго - 70h

outp(0x21, 0x04); //ICW3 - номер выхода ведущего
контроллера, к которому подсоединен ведомый (00000 100)

outp(0x21, 0x01); //ICW4 - настройка дополнительных ( 000
0 00 0
1)
// режимов работы должны быть
спец. вложенный режим режим работы использование
режима поддержка типа

```

```

//          обычный режим          установлены в
0      (не использовать)      (00 - небуфферизованный автоматического
завершения      процессора (8086)
//
//          режим)          прерывания (не
используется)

    outp(0xA0, 0x11);    //ICW1 - инициализация ведомого
контроллера
    outp(0xA1, 0x08);    //ICW2 - базовый вектор ведомого
    outp(0xA1, 0x02);    //ICW3 - номер выхода ведомого
контроллера, к которому подсоединен ведущий      (выход 2)
    outp(0xA1, 0x01);    //ICW4 - обычный режим

    _enable();          // Разрешить прерывания
}

int main()
{
    unsigned far *fp;
    initialize();          // Функция
инициализации контроллеров прерываний
    system("cls");

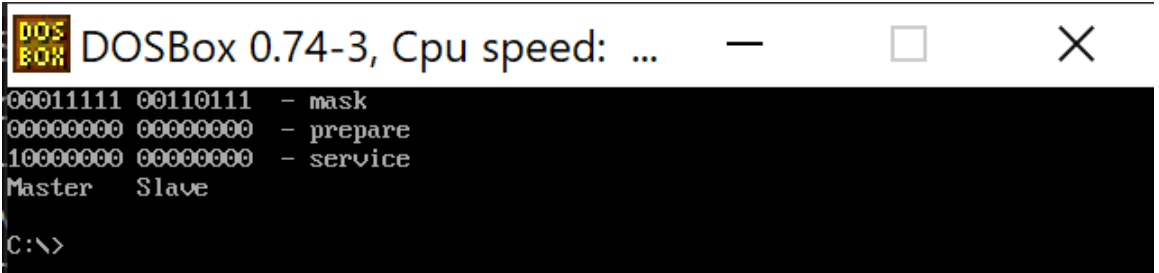
    printf("          - mask\n");
    printf("          - prepare\n");
    printf("          - service\n");
    printf("Master    Slave\n");

    FP_SEG(fp) = _psp;          // FP_SEG
возвращает сегмент дальнего указателя
    FP_OFF(fp) = 0x2c;          // FP_OFF
возвращает смещение дальнего указателя
    _dos_freemem(*fp);          // Освобождаем
память, которая выделена под far-указатель (дальний указатель)

    _dos_keep(0, (_DS - _CS) + (_SP / 16) + 1);    // Оставляем
программу резидентной (первый параметр - код завершения,
// второй параметр -
объём памяти, который должен быть зарезервирован для программы после
завершения)
    return 0;
}

```

Тестовые примеры



```

DOS
BOX
DOSBox 0.74-3, Cpu speed: ...
00011111 00110111 - mask
00000000 00000000 - prepare
10000000 00000000 - service
Master Slave
C:\>

```

Рисунок 4.1 - Результат работы программы при запуске

4. Заключение

При выполнении данной лабораторной работы была разработана программа, выполняющая перенос векторов прерываний ведущего и ведомого контроллера на пользовательские прерывания.

Программа была скомпилирована при помощи BORLANDC в DosBox.